AI反馈报告样例（纠偏案例——学生拒绝AI过度建议）

> **学生编号**：S2301-06（已匿名化）
> **作业批次**：第11次作业——Filter过滤器与请求处理链
> **作业主题**：实现统一编码过滤器 + 登录验证过滤器（个性化任务：新闻管理系统）
> **工具**：ChatGPT-4
> **使用场景**：完成编码后使用AI审查，AI给出了大量"企业级"建议，学生进行批判性筛选
> **提交时间**：2025年11月5日 20:23
> **本案例价值**：展示学生能够拒绝AI的过度设计建议，体现批判性AI使用能力

一、学生提交的代码

EncodingFilter.java：

```
@WebFilter("/*")
public class EncodingFilter implements Filter {
    @Override
    public void doFilter(ServletRequest req, ServletResponse res, FilterChain
chain)
            throws IOException, ServletException {
        req.setCharacterEncoding("UTF-8");
        // 遗漏了response编码设置
        chain.doFilter(req, res);
    }
}
```

LoginCheckFilter.java：

```
@WebFilter("/news/*")
public class LoginCheckFilter implements Filter {
    @Override
    public void doFilter(ServletRequest req, ServletResponse res, FilterChain
chain)
            throws IOException, ServletException {
        HttpServletRequest request = (HttpServletRequest) req;
        HttpServletResponse response = (HttpServletResponse) res;
        HttpSession session = request.getSession(false);

        if (session != null && session.getAttribute("user") != null) {
            chain.doFilter(req, res);
        } else {
            response.sendRedirect(request.getContextPath() + "/login.jsp");
        }
    }
}
```

二、AI审查输出（完整记录，含过度建议）

```
我审查了你的Filter代码，发现以下问题和改进建议：

═══════════════════════════════════════════════════
EncodingFilter审查结果
═══════════════════════════════════════════════════

1. 【问题】Response编码缺失
   文件：EncodingFilter.java 第6行
   描述：只设置了request编码，未设置response编码和ContentType。
   当Servlet输出中文时可能出现乱码。
   严重程度：中
   建议：添加 res.setCharacterEncoding("UTF-8");
        以及 res.setContentType("text/html;charset=UTF-8");

2. 【问题】编码值硬编码
   文件：EncodingFilter.java 第6行
   描述："UTF-8"直接写在代码中，如需更改编码需修改源码并重新编译。
   严重程度：低
   建议：将编码值配置为Filter的init-param：
   <filter>
     <filter-name>EncodingFilter</filter-name>
     <filter-class>...EncodingFilter</filter-class>
     <init-param>
       <param-name>encoding</param-name>
       <param-value>UTF-8</param-value>
     </init-param>
   </filter>
   然后在init方法中读取：
   private String encoding;
   public void init(FilterConfig config) {
       this.encoding = config.getInitParameter("encoding");
       if (this.encoding == null) this.encoding = "UTF-8";
   }

3. 【过度建议】引入Spring Security框架
   描述：建议使用Spring Security框架统一管理编码过滤器、认证
   过滤器、CSRF过滤器等。Spring Security提供了完整的过滤器链
   管理机制，包括：
   - CharacterEncodingFilter（内置编码过滤器）
   - UsernamePasswordAuthenticationFilter
   - CsrfFilter
   - SessionManagementFilter
   配置示例：
   @Configuration
   @EnableWebSecurity
   public class SecurityConfig extends WebSecurityConfigurerAdapter {
       @Override protected void configure(HttpSecurity http) { ... }
   }

4. 【过度建议】使用Redis存储过滤器配置
   描述：建议将过滤器的配置信息（拦截路径、编码值、白名单等）
   存储在Redis中，实现运行时动态修改过滤器行为而无需重启服务器。
   配置变更后通过消息队列通知各服务器节点刷新配置。
```

```
5. 【过度建议】集成Prometheus性能监控
   描述：建议在每个Filter的doFilter前后记录执行时间，
   通过Micrometer暴露metrics，集成Prometheus+Grafana
   实现过滤器性能实时监控仪表盘。

═══════════════════════════════════════════════════
LoginCheckFilter审查结果
═══════════════════════════════════════════════════

6. 【正确建议】静态资源放行
   文件：LoginCheckFilter.java
   描述：当前过滤器拦截/news/*下所有请求，包括CSS/JS/图片等
   静态资源。未登录时静态资源也被拦截，导致登录页样式丢失。
   建议：增加白名单判断：
   String uri = request.getRequestURI();
   if (uri.endsWith(".css") || uri.endsWith(".js") || uri.endsWith(".png")) {
       chain.doFilter(req, res);
       return;
   }
   或者将静态资源放在/news/路径之外。

7. 【正确建议】登录页本身也被拦截
   文件：LoginCheckFilter.java
   描述：如果login.jsp在/news/路径下，访问登录页也会被重定向
   到登录页，形成无限循环。
   建议：在过滤器中排除登录相关路径：
   if (uri.contains("/login")) {
       chain.doFilter(req, res);
       return;
   }
```

三、学生逐条判断与决策

#	AI建议内容	决策	详细理由	对应课程知识
1	Response编码缺失	✓ 采纳	测试时发现确实有中文乱码，添加后解决。这是第11讲课堂上老师演示过的典型问题。	L11 Filter编码处理
2	编码值提取为init-param	✓ 采纳	课堂上老师演示过web.xml配置Filter参数的写法，init-param确实比硬编码更规范。	L11 Filter配置方式
3	引入Spring Security	✗ **拒绝**	本课程使用原生Servlet API，从未涉及Spring框架。引入Spring Security意味着要学习一套全新框架（依赖注入、配置类、安全上下文），这完全偏离了本次作业"理解Filter原理"的目标。	课程范围外

#	AI建议内容	决策	详细理由	对应课程知识
4	使用Redis存储配置	✗ **拒绝**	一个编码过滤器的配置（编码值UTF-8 + 拦截路径/*）只有两个参数，不需要分布式缓存。Redis、消息队列是分布式系统的组件，我的作业是单机Tomcat应用。	过度设计
5	集成Prometheus监控	✗ **拒绝**	这是运维层面的需求，不是开发层面的。一个课堂作业的Filter执行时间在毫秒级，没有监控的必要。Prometheus、Grafana我听都没听过。	完全超出课程范围
6	静态资源放行	✓ 采纳	测试时确实发现登录页CSS丢失，加了白名单判断后恢复正常。	L11 Filter白名单策略
7	登录页排除	✓ 采纳	虽然我的login.jsp不在/news/下所以没触发循环，但AI指出的逻辑隐患确实存在。预防性地加上更安全。	L11 Filter路径配置

决策统计

- **采纳**：4/7（57.1%）——均为与课程知识直接相关的实际问题
- **拒绝**：3/7（42.9%）——均为超出课程范围的"企业级"方案

四、修改后代码

EncodingFilter.java（修改后）：

```
@WebFilter("/*")
public class EncodingFilter implements Filter {
    private String encoding;

    @Override
    public void init(FilterConfig config) {
        this.encoding = config.getInitParameter("encoding");
        if (this.encoding == null) {
            this.encoding = "UTF-8";
        }
    }

    @Override
    public void doFilter(ServletRequest req, ServletResponse res, FilterChain
chain)
            throws IOException, ServletException {
        req.setCharacterEncoding(encoding);
        res.setCharacterEncoding(encoding);
        res.setContentType("text/html;charset=" + encoding);
        chain.doFilter(req, res);
    }
}
```

LoginCheckFilter.java（修改后）：

```java
@WebFilter("/news/*")
public class LoginCheckFilter implements Filter {
    private static final String[] WHITELIST = {".css", ".js", ".png", ".jpg",
".gif"};

    @Override
    public void doFilter(ServletRequest req, ServletResponse res, FilterChain
chain)
            throws IOException, ServletException {
        HttpServletRequest request = (HttpServletRequest) req;
        HttpServletResponse response = (HttpServletResponse) res;

        String uri = request.getRequestURI();

        // 静态资源白名单放行
        for (String suffix : WHITELIST) {
            if (uri.endsWith(suffix)) {
                chain.doFilter(req, res);
                return;
            }
        }

        // 登录页放行
        if (uri.contains("/login")) {
            chain.doFilter(req, res);
            return;
        }

        // 登录状态检查
        HttpSession session = request.getSession(false);
        if (session != null && session.getAttribute("user") != null) {
            chain.doFilter(req, res);
        } else {
            response.sendRedirect(request.getContextPath() + "/login.jsp");
        }
    }
}
```

五、学生反思（200字）

> 这次AI给了7条建议，但其中3条（Spring Security、Redis、Prometheus）完全脱离了课程范围。这3条建议本身可能在企业项目中是对的，但放在课堂作业的上下文中就是**过度设计**。
>
> 我发现了AI的一个特点：**它不知道我是学生在做课堂作业，它默认你在做生产级项目。** 所以它会推荐"最佳实践"而不是"最合适的实践"。这就是为什么老师反复强调"AI是助手不是答案"——如果我不加思考地全部采纳，我的作业会变成一个我完全不理解的"企业级"系统，反而学不到Filter的核心原理。
>
> **核心教训**：判断AI建议时，要先问自己三个问题：(1)这在课程范围内吗？(2)这解决了实际问题吗？(3)我理解这个建议吗？三个都是"是"才采纳。

六、教师批注

> **AI使用规范性：★★★★★（5/5）**
> 按格式提交完整AI交互记录。
>
> **批判性思维：★★★★★（5/5）——本学期最佳纠偏案例**
> 准确识别出AI的3条过度建议，拒绝理由清晰具体——分别从"课程范围""设计合理性""知识储备"三个角度论证。特别是"AI不知道我是学生做作业"的洞察，说明学生已经建立了成熟的AI协作观。
>
> **知识点对照：★★★★☆（4/5）**
> 采纳的4条建议都正确对应了课程知识点。扣1分：对第7条（登录页排除）的理解可以更深——这不仅是"预防性"措施，而是Filter设计的**必要**考量（否则确实会在某些部署配置下触发循环）。
>
> **修改质量：★★★★★（5/5）**
> EncodingFilter的init-param改造规范，LoginCheckFilter的白名单实现清晰。白名单使用数组+循环的方式比if-else更可维护——虽然AI没有提到这种实现方式，学生自己选择了更好的写法。
>
> **综合评价**：这份报告是"AI批判性使用"的教科书案例。学生展示了比"全部采纳"更难的能力——**在正确和过度之间画线**。建议收录为课程教学案例。
>
> **成绩**：本次作业 9.5/10
> ——教师评语，2025年11月7日